Land Use Optimization with the NSGA-III Algorithm

Kyler Kasravi

March 2026

Abstract

Urban land use planning requires balancing many competing objectives, a challenge known as a multi-objective optimization problem. In this paper, we develop a grid-based land use optimization method using the Non-dominated Sorting Genetic Algorithm III (NSGA-III). Our approach assigns land use types (residential, commercial, industrial, essential services, and green space) to cells on a discrete grid, optimizing four objectives. Our objectives are: minimizing job commute distance, minimizing essential service commute distance, maximizing livability, and minimizing industrial-residential adjacency. Buildings are modeled as standalone cells for residential and green space, and as multi-cell clusters for commercial, essential, and industrial building types. The shape and placement of these building clusters are evolved through genetic operators. We implemented the framework in Python using the PyMOO library and created an interactive GUI for configuring and visualizing the optimization process. Experiments on grids of varying sizes demonstrated that the algorithm consistently achieved livability scores within approximately 3% of the cluster-constrained theoretical maximum, with commute distances and industrial adjacency reflecting Pareto trade-offs, as they are minimized in opposition to each other. Our work was inspired by the approach of Sicuaio et al. (2024), who applied the NSGA-III algorithm to land use planning in Maputo, Mozambique; we extend that approach by introducing free-form building growth, nearest cell-distance computations, and proportional grid scaling.

1 Introduction

Land use planning is vital to social and economic development. As population grows, the question of how to allocate land use types across a city—where to place housing, parks, commercial districts, industrial zones, and essential services—becomes an important problem impacting the daily lives of city residents. A well-planned city can promote walkability, economic productivity, and resilience.

The challenge in our project is that urban planning objectives are inherently conflicting. A layout that minimizes commute distance by clustering jobs near housing may also reduce livability, because industrial facilities end up near residential areas. Maximizing green space improves livability at the cost of commute distance. No single layout can optimize all objectives, making this problem a multi-objective optimization problem.

Multi-objective optimization problems seek solutions that represent the best trade-offs among competing objectives. The set of such solutions, in which no solution can improve on one objective without worsening another, is called the Pareto front; we state this definition formally in Definition 2.1 of Section 2. Rather than producing a single "best" answer, a multi-objective optimizer gives its users a selection of Pareto-optimal solutions, allowing them to select a different prioritization of objectives.

1.1 Motivation

This project is motivated by the work of Sicuaio, Zhao, Pilesjö, Shindyapin, and Mansourian [2], who applied a specialized version of the NSGA-III algorithm to land use allocation in the KaMavota district of Maputo, Mozambique. Their study optimized five different objectives—economic value, carbon emissions, accessibility, space syntax integration, and compactness—across a massive grid of 2,348 cells generated from real geographic data at 50 m resolution. They showed that their application of the NSGA-III algorithm outperformed the standard NSGA-III in both solution quality and computational efficiency.

Our project borrows this idea of applying the NSGA-III to grid-based land use optimization and takes it in several new directions. Rather than working with real data and fixed land use proportions, we built a configurable grid where the user can specify grid size, building types, sizes, and objective weights. This allows exploration of the solution space and of how the algorithm behaves under different configurations.

1.2 Project Evolution

Our project progressed through multiple phases. In the first phase, we implemented a simple fraction-based model whose decision variables were continuous proportions: x_1 for residential, x_2 for green space, and x_3 for industrial, where $x_1 + x_2 + x_3 = 1$. Our objectives were defined as:

$$f_1(\mathbf{x}) = 200x_1 + 50x_2 + 300x_3 \quad \text{(cost)}, \tag{1}$$

$$f_2(\mathbf{x}) = |x_1 - x_3| \quad \text{(commute imbalance)}, \tag{2}$$

$$f_3(\mathbf{x}) = -(1.5x_2 - 0.2|x_1 - 0.4| - 0.2|x_3 - 0.3|) \quad \text{(negative livability)}. \tag{3}$$

This model served as a proof of concept, showing that the base NSGA-III algorithm could find a Pareto front for a land use problem, but it lacked any visual or spatial representation.

The second phase of the project was moving to a discrete grid. We applied the algorithm to a 4×4 discrete grid where each cell was assigned one of five land use types: residential, green space, commercial, industrial, or essential. Distances were computed using taxicab geometry, that is, along the grid, by summing the absolute horizontal and vertical differences between two points. This reflects the grid-like structure of most cities, removing the need for a road network. This phase introduced spatial objectives, where commute distance was the average taxicab distance from residential cells to job cells, and livability became a function determined by the type of each residential cell's neighbors.

In the third phase, we scaled our grid to a 20×20 grid (400 cells) with multi-cell buildings that can take any connected shape. Industrial buildings occupied 9 cells each, commercial and essential buildings occupied 4 each, and residential and green space land use types occupied single cells. This allowed the algorithm to evolve building placement and shape, producing a more diverse solution space through a growth-based encoding in which directional genes determine how a building expands from its anchor point.

Finally, in the fourth and most recent phase of the project, we implemented a GUI allowing for the changing of objective weights, scaling of the grid to different sizes, and changing of building proportions, numbers, and sizes. Additionally, the algorithm was enhanced by vectorizing the commute calculations, allowing for faster runtimes. This allowed a deeper exploration of the solution space and better evaluations, through running more simulations with larger numbers of generations. We compared taxicab versus Euclidean commute distances, and we were able to find and display new solutions on the Pareto front by interpolating new points between existing Pareto-optimal solutions. Below is an example of a taxicab geometry distance calculation.

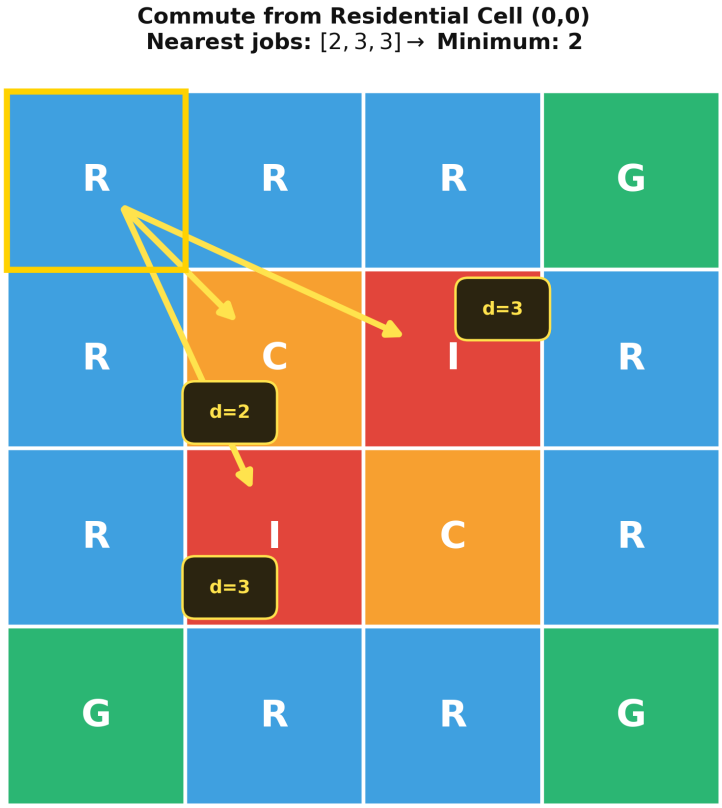


Figure 1: Taxicab geometry distance calculation example. Figure generated with the assistance of Anthropic's Claude [1].

2 Background

Definition 2.1 (The Pareto Front)**.** A feasible solution A is Pareto optimal if, for every other feasible solution X, A is no worse than X in all objectives and is strictly better in at least one. The set of all Pareto-optimal solutions is called the Pareto set, and the image of the Pareto set in objective space is called the Pareto front.

Consider a city layout with two objectives: commute distance (minimized) and livability (maximized). If layout A has a commute of 3.0 and a livability of 390, and layout B has a commute of 2.5 and a livability of 370, then neither layout dominates the other: A has better livability, while B has the shorter commute. Given that no other feasible layout improves on either of them, both A and B are Pareto optimal.

2.1 Evolutionary Algorithms and Genetic Operators

Evolutionary algorithms are population-based optimization strategies inspired by biological evolution [3, 4]. They maintain a population of candidate solutions that evolve in parallel through selection, crossover, and mutation.

Each candidate solution is encoded as a chromosome, a vector of values called genes. The encoding maps the chromosome to a solution in the decision space through a decoding function.

Our key genetic operators are:

Selection: Parents are chosen from the current population based on Pareto rank.

Crossover: Two parent chromosomes exchange values to produce offspring. For example, in a single-point crossover, a random position p is chosen, and the resulting offspring inherits genes $1, \ldots, p$ from one

parent and genes $p+1, \ldots, n$ from the other.

Mutation: Random changes are applied to individual genes with some probability. This introduces genetic variation and helps the algorithm escape local minima or maxima that may be preventing the creation of more optimal solutions.

2.2 The NSGA-III Algorithm

The Non-dominated Sorting Genetic Algorithm III (NSGA-III) [5] is an evolutionary algorithm designed for many-objective optimization problems (those with four or more objectives). It extends the NSGA-II algorithm [6] by adding a reference-point-based selection strategy.

The NSGA-III procedure operates as follows:

1. Initialization: generate an initial population of N random solutions and a set of reference points in m-dimensional objective space.
2. Genetic operations: apply selection, crossover, and mutation to create an offspring population of size N (equal to the starting population).
3. Combination: merge the parent and offspring populations into a combined population of size $2N$.
4. Non-dominated sorting: sort the combined population into fronts $F_1, F_2, \ldots$, where F_1 contains all non-dominated solutions, F_2 contains solutions dominated only by those in F_1, and so on.
5. Reference-point-based selection: fill the next-generation population front by front. When a front must be split—that is, when including it in its entirety would exceed the population size N—solutions are selected based on their association with reference points, preferring solutions closest to reference directions that are underrepresented in the population thus far.
6. Completion: steps 2–5 are repeated until the number of generations specified by the user is reached.



Figure 2: NSGA-III algorithm flowchart. Steps 2 and 3 are covered within the evaluate objectives box. Figure generated with the assistance of Anthropic's Claude [1].

3 Objective Functions

We optimize four objectives simultaneously. Throughout this section, a layout is represented by a grid in which each cell stores an integer code for its land use type, and we write grid$[r][c]$ for the code stored in row r and column c. The correspondence between codes and land use types is fixed as follows: 0 is residential, 1 is green space, 2 is commercial, 3 is industrial, and 4 is essential services. Cells of type 2 and 3 are together referred to as job cells. We also write $N_t(r, c)$ for the number of the four direct neighbors of cell (r, c) whose land use type is t.

3.1 Job Commute Distance

Let $R = \{(r, c) : \text{grid}[r][c] = 0\}$ be the set of residential cells and $J = \{(r, c) : \text{grid}[r][c] \in \{2, 3\}\}$ be the set of job cells (commercial and industrial). The job commute objective is:

$$f_1 = \frac{1}{|R|} \sum_{(r_i,c_i)\in R} \min_{(r_j,c_j)\in J} d_T\big((r_i, c_i),\, (r_j, c_j)\big). \tag{4}$$

This is the mean of the minimum taxicab distances from each residential cell to its nearest job cell. Here we compute the distance to the nearest individual cell rather than to the centroid of a cluster; using the centroid was an issue we faced earlier in the project, and it led to inflated commute distances.

3.2 Essential Commute Distance

Similarly, let $E = \{(r, c) : \text{grid}[r][c] = 4\}$ be the set of essential service cells. The essential commute objective is:

$$f_2 = \frac{1}{|R|} \sum_{(r_i,c_i)\in R} \min_{(r_e,c_e)\in E} d_T\big((r_i, c_i),\, (r_e, c_e)\big). \tag{5}$$

This is calculated just like the job commute distance. The two objectives were separated to ensure that the algorithm would place essential cells in the most optimal positions for whole-grid commute, since calculating them together with job commute would lead to poorer essential commute distances.

3.3 Livability

The livability score is computed per residential cell based on the types of its four direct neighbors, using the neighbor counts $N_t(r, c)$ defined at the start of Section 3. The total livability is then calculated as:

$$f_3 = -\sum_{(r,c)\in R} \Big(1.0 + 0.5 \cdot N_1(r, c) + 0.3 \cdot N_4(r, c) - 0.1 \cdot N_2(r, c) - 2.0 \cdot N_3(r, c)\Big). \tag{6}$$

Each residential cell has a base score of 1.0, with bonuses for adjacent green space (+0.5) and adjacent essential cells (+0.3), a small penalty for adjacent commercial cells (−0.1), and a larger penalty for industrial adjacency (−2.0).

3.4 Industrial Adjacency

The fourth objective counts the number of residential cells with at least one industrial neighbor:

$$f_4 = \big|\{(r, c) \in R : N_3(r, c) > 0\}\big|. \tag{7}$$

This objective serves as further motivation for the algorithm to separate industrial and residential zones, and it consistently pushes industrial zones to the border of our grids, mimicking how industrial areas are often located far away from residential ones.

3.5 Constraint: Building Collisions

Buildings often fail to grow and evolve to their target size and shape due to collisions with previously placed buildings. The number of these collisions is recorded as a constraint violation:

$$g_1 = (\text{number of buildings that failed to reach target size}) \leq 0. \tag{8}$$

The algorithm handles this issue by preferring feasible solutions over infeasible ones (collisions) when selecting solutions to further evolve.

4 PyMOO Framework and NSGA-III Changes

All of our implementations use PyMOO [7], an open-source Python library for multi-objective optimization. PyMOO provides the NSGA-III algorithm along with the supporting reference direction generation via the Das–Dennis method [8], non-dominated sorting, and the reference-point niching that decides which solutions survive when a front must be split. Instead of implementing the algorithm from scratch, we treated the PyMOO NSGA3 class as a solver and adapted our problem to it.

PyMOO separates a problem into four components: the problem, which sets the decision variables, objectives, and constraints; and the sampling, crossover, and mutation operators. Each of these is a Python class that we pass into the algorithm at construction time.

4.1 From Continuous Fractions to a Discrete Grid

The transition from the fraction-based test case to the discrete grid required a change in what a chromosome is. In the first phase of the project, a chromosome was a vector (x_1, x_2, x_3) with continuous proportions, and PyMOO's real-valued operators, simulated binary crossover and polynomial mutation, were used directly. This was enough to prove that the NSGA-III algorithm could produce a Pareto front for our toy problem, but a vector of three fractions cannot describe where anything is placed on a 2D grid.

Moving to a grid meant the chromosome had to encode an entire 2D grid layout. The first approach, assigning one gene per cell to hold its land use type, fails for two reasons. First, a 20×20 grid would have a search space of $5^{400} \approx 4 \times 10^{279}$ possible layouts, a number far greater than the number of atoms in the observable universe. Second, a per-cell encoding cannot ensure that the cells of a single building stay connected; applying the crossover operation would tear buildings apart, producing invalid layouts. We implemented a compact, growth-based integer encoding, which is described below, and replaced PyMOO's real-valued operators with integer operators.

4.2 Cluster-Growth Chromosome Encoding

Each multi-cell building (industrial, commercial, essential) is encoded by a small block of integer genes. We start with an anchor gene pair (r, c) marking the building's starting cell, followed by a sequence of growth genes, each an integer modulo 4 encoding one of four directions: up, right, left, and down. A building of target size s uses $2 + (s - 1)$ genes. The building is decoded by placing its anchor and then repeatedly attempting to grow into the cell indicated by the next growth gene; if that cell is occupied or off-grid, the decoder uses the first available cardinal direction so that growth can succeed. The cells left over after all clusters are placed are filled by single-cell residential and green space types, determined by the user's choice of green space and residential proportions.

An advantage of this encoding is that connectivity is guaranteed: because every cell of a building is grown adjacent to an existing cell of that building, the building cluster is always connected regardless of how crossover and mutation change the genes. This is the main reason we abandoned an earlier anchor-point labeling scheme in favor of growth-based encoding. The total encoding length for one grid is

$$L = n_{\text{ind}}(2 + s_{\text{ind}} - 1) + n_{\text{com}}(2 + s_{\text{com}} - 1) + n_{\text{ess}}(2 + s_{\text{ess}} - 1) + (\text{grid size})^2, \tag{9}$$

where n and s are the count and size of each building type. For the default 20×20 configuration, this gives a chromosome of just a few hundred integers, much smaller than the per-cell encoding while still describing a full layout.

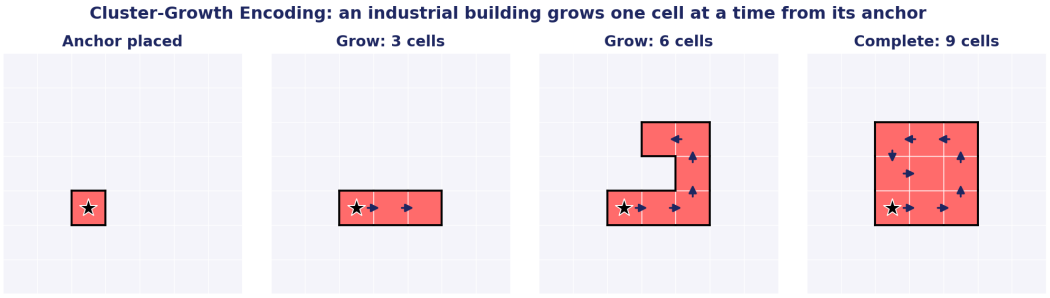


Figure 3: Cluster-growth encoding. Each building of industrial, commercial, or essential class type is described by an anchor gene pair and a sequence of directional growth genes that grow the building one cell at a time, always producing a connected cluster. Figure generated with the assistance of Anthropic's Claude [1].

4.3 Custom Integer Operators

Because the encoding is integer-valued, we replaced PyMOO's default operators with three custom classes:

- **Sampling** (`Integer Sampling`): the initial population is drawn uniformly at random from the integer range $[0, x_u]$, where x_u is one less than the grid size, to ensure that anchors land on the grid and that direction genes still cover all four directions.
- **Crossover** (`Integer Crossover`): a single-point crossover is applied with probability 0.9. A cut point is chosen at random along the gene vector, and the two offspring inherit complementary segments of their parents' gene vectors. Because connectivity is enforced during decoding rather than encoding, crossover never produces an invalid building, and in the event that swapped anchor points cause a collision, the invalid layout is removed through the collision constraint.
- **Mutation** (`Integer Mutation`): each gene is mutated with probability 0.2 by adding a small random integer offset drawn from $[-\eta, \eta]$ with $\eta = 4$. This moves anchors and growth directions slightly rather than completely randomizing them, allowing refinement of a layout while still permitting larger jumps over many generations. This is the reason commute distance tends to take longer to settle: these mutations require many generations to produce large changes.

4.4 Problem Definition and Constraint Handling

The problem itself is defined as a PyMOO Problem subclass with L integer decision variables, four objectives, and one inequality constraint. For each individual in the population (120), the decoder builds the grid, the four objective functions are evaluated, and the number of buildings that did not reach their target size is recorded as the constraint value g_1. NSGA-III handles this constraint through its selection mechanism: when the algorithm selects the 120 solutions to keep, feasible solutions (those with $g_1 = 0$) are always preferred over infeasible ones, and among infeasible solutions those with smaller constraint violations are preferred. In every run we recorded, the population became fully feasible within the first few generations, after which the search was driven by the four objectives. This approach differs from Sicuaio et al. [2], who modified the crossover and mutation operators to preserve building sizes. To keep our operators simple, we allow infeasible solutions in the beginning, which are pruned away as the algorithm runs.

4.5 The Interactive GUI

The interactive GUI was built in Python using the Tkinter library. It produces a control panel that lets the user adjust grid size, generation count, population size, building counts and sizes, and the four objective weights, then launches a run of the algorithm and renders the resulting Pareto scatter plots and best-per-objective layouts without the user editing any code. The GUI was developed with the assistance of

Anthropic's Claude [1], which generated the Tkinter layout, the event handling that connects the controls to the PyMOO solver, and the Matplotlib embedding used to display layouts and convergence plots; this code was then reviewed, tested, and integrated.

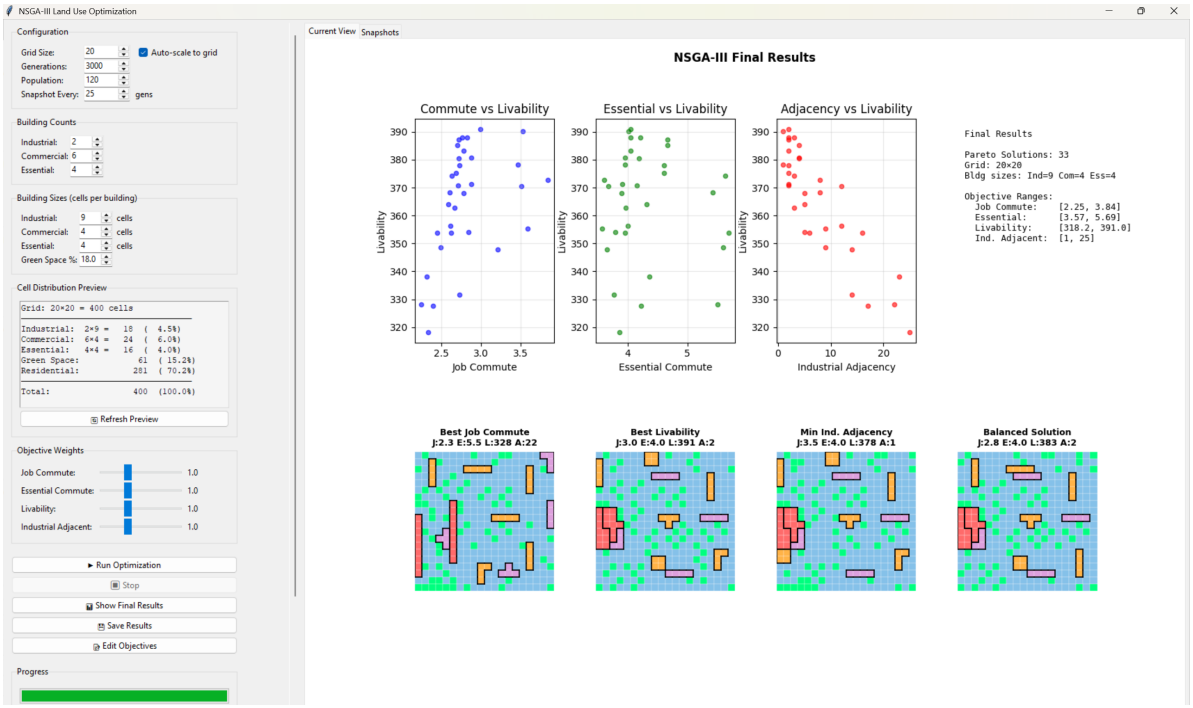


Figure 4: The interactive GUI. The left panel exposes grid size, generation count, population, building counts and sizes, and objective-weight sliders; the right panel shows Pareto scatter plots and snapshots the most balanced grid across the four objectives every 25th generation. When a run finishes, the GUI displays four layouts: best job commute, best livability, minimum industrial adjacency, and a balanced solution.

5 Solution Space and Visuals

A 20×20 grid has 400 cells, each of which can take one of five land use types, giving $5^{400} \approx 4 \times 10^{279}$ possible raw layouts. The cluster-growth encoding reduces the effective dimensionality from 400 free cells to a few hundred structured integers, but the difficulty of working in such a large space is what motivates an evolutionary approach in the first place. The NSGA-III algorithm samples this space with a population, keeps the non-dominated layouts, and recombines them, gradually concentrating its search in objective-aligned regions and vastly narrowing this search space down.

5.1 Proportional Grid Scaling

To compare behavior across grid sizes, the building configuration is scaled proportionally from the 20×20 baseline. Given an arbitrary grid size, the total number of cells devoted to each building type is scaled by the ratio of total cells, and this scaling chooses a building size and count whose product best matches that target. Green space is then allocated as a fraction (about 18%) of the cells left after the clusters are placed, with residential filling the remainder. This keeps the land use proportions roughly constant as the grid grows or shrinks, and exactly constant for certain carefully chosen grid sizes, so that we can scale results from smaller grids, where the best layout is known, up to larger grids, where finding this specific layout for each objective is not feasible with the computing power we have access to.

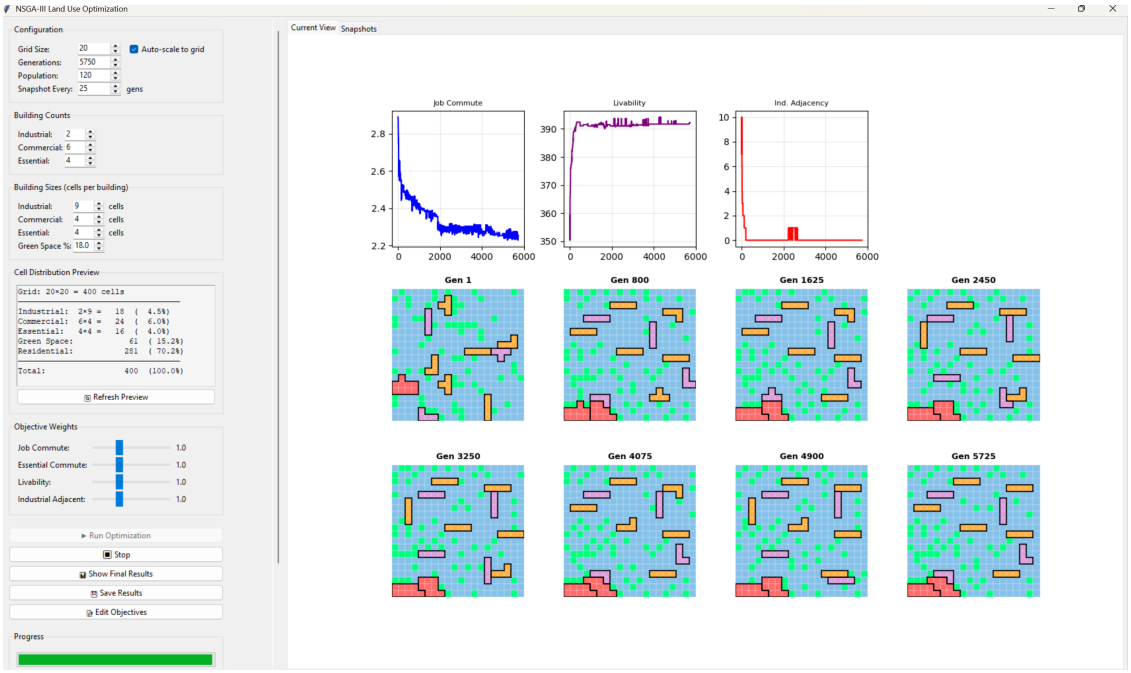


Figure 5: Per-objective convergence over a long run. Industrial adjacency reaches zero within the first few hundred generations, and livability peaks early, making minor improvements at the cost of commute distance; these improvements often do not survive niching. Job commute, meanwhile, continues to make small improvements out past generation 2,000.

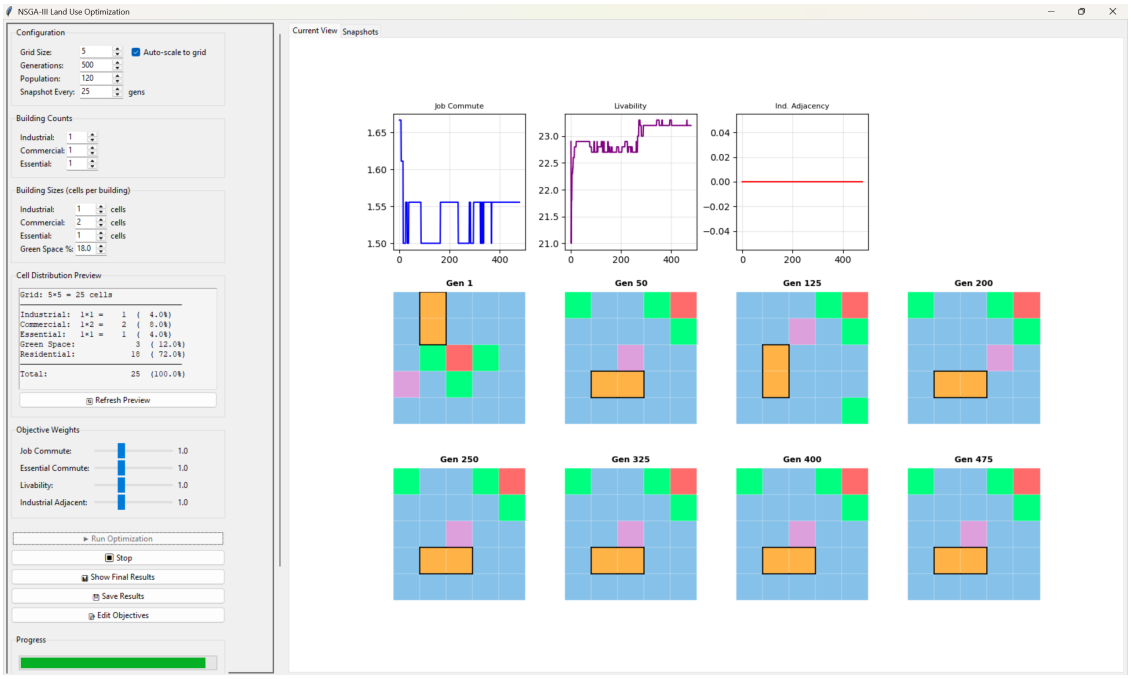


Figure 6: A 5×5 test case, shown as snapshots taken every 25 generations of a 500-generation run. The population settles on the same family of Pareto-optimal layouts, sometimes rotated or reflected, and holds it from roughly generation 250 onward, showing that the algorithm finds the global Pareto set rather than a local one for this test case.

5.2 What the Algorithm Discovers

Watching the evolution unfold, a few behaviors emerge consistently. Industrial buildings are pushed to the corners and edges of the grid, since the industrial-adjacency objective and the heavy -2.0 livability penalty both punish any residential cell next to a factory. Green space disperses into a scattered, checkerboard distribution rather than clumping, because each green cell only benefits the residential cells immediately adjacent to it, so spreading green out raises livability for more homes. Commercial and essential clusters distribute themselves across the grid, axis-aligned and close to residential cells, to shorten the average commute, since a centralized cluster would leave residents on the far side of the grid with long trips. None of these behaviors were programmed in; they emerge from the interaction of the four objectives, which is one of the more compelling aspects of the result.

6 Efficacy of the Algorithm

Early in the project, the commute objectives measured the distance from each residential cell to the centroid of each building cluster, rather than to the nearest individual cell of that cluster. This overestimated commute distances, sometimes badly: a residential cell sitting directly adjacent to the corner of a cluster could be assigned a distance of 3 (to the centroid) when its true distance to the nearest cell of that building was 1. Correcting this required rewriting the commute calculation to compute, for every residential cell, the minimum taxicab distance to any job cell, vectorized with NumPy:

```
def calc_job_commute(grid):
    res_coords = np.argwhere(grid == 0)
    job_coords = np.argwhere((grid == 2) | (grid == 3))
```

```
    diff = np.abs(res_coords[:, None, :] - job_coords[None, :, :])
    dists = diff[:, :, 0] + diff[:, :, 1]
    return float(dists.min(axis=1).mean())
```

Beyond producing better objective values, this also produced a 5–8× speedup over the previous loop-based implementation, because the entire matrix of pairwise distances is computed in compiled NumPy code rather than in a Python loop. The same vectorization was applied to the livability and adjacency objectives, which count neighboring cell types using array shifts rather than per-cell neighbor look-ups.

6.1 Taxicab Distance

The taxicab distance between two points (x_1, y_1) and (x_2, y_2) on a grid is [9]:

$$d_T\big((x_1, y_1), (x_2, y_2)\big) = |x_1 - x_2| + |y_1 - y_2|. \tag{10}$$

In contrast, the Euclidean distance is $d_E = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. For randomly distributed points on a grid, the expected ratio d_T/d_E approaches $4/\pi \approx 1.273$. An interesting finding of our project is that the NSGA-III algorithm consistently produces layouts where this ratio is significantly below $4/\pi$, going as low as approximately 1.10, because the optimizer preferentially places buildings so that they are axis-aligned, which minimizes taxicab distances.

6.2 Theoretical Ceilings on Livability

Rather than judging the algorithm only against its own previous runs, we compute how close it gets to the best livability that is achievable on the 20×20 grid given our objectives. There are three limits we computed for a 20×20 grid. The theoretical maximum ($\approx$ 422–430) assumes every residential cell could be surrounded by ideal neighbors with no clustering constraints at all. The cluster-constrained maximum ($\approx$ 406) accounts for the fact that buildings must be connected clusters of fixed size, which forces some adjacent spaces. The best observed value from NSGA-III lands around 395–398. Expressed as a fraction of the cluster-constrained achievable maximum, the algorithm reaches roughly 97% of the maximum attainable value.

The remaining gap of about 3% is not a failure of the algorithm. It is the cost of our optimization: a solution that maximized livability alone would get closer to the ceiling, but it would do so at the cost of commute distance and the other objectives. The non-dominated solutions we find trade a few points of livability for better performance elsewhere, which is the entire goal of producing a Pareto front rather than a single answer.

6.3 Convergence Behavior

The four objectives converge at very different rates, which is visible when a single run is extended to thousands of generations.

- **Industrial adjacency** usually reaches zero within the first few hundred generations; once industrial clusters are pushed to the edges, there is little reason for them to move back, since reducing the number of residential cells adjacent to industry is both its own objective and encouraged by the livability objective through penalties.
- **Livability** rises quickly and peaks early, settling near its achievable maximum (a livability score of roughly 395–398) within the first few hundred generations, often making small improvements later at the cost of commute distance.
- **Job commute** is the slowest objective. It drops sharply over roughly the first thousand generations and then continues to make small improvements, generally settling around generation 2,000.
- **Essential commute** converges more slowly than livability and adjacency, in part because there are fewer essential cells, so each placement decision has a larger effect on the average.

The consequence is that very long runs mostly buy small refinements in commute distance; the other three objectives are essentially settled long before the run ends.

6.4 Cost of Population Size and Sorting

The non-dominated sorting employed by the NSGA-III algorithm has worst-case complexity that grows quadratically in the population size, which might suggest that larger populations would become prohibitively expensive. In practice, at the population sizes we use (84–120), the cost is dominated not by sorting but by evaluation. Since decoding a chromosome into a grid and computing its four objectives takes roughly 3 ms per individual on the 20×20 grid, this per-individual cost so heavily outweighs the sorting cost that the scaling with population size is much less significant. Moving from the default population of 120 to 168 roughly doubles the per-generation time and requires on the order of 45–65% more total wall-clock time to reach similar solution quality. Overall, for this problem, evaluation itself is expensive, and the theoretical quadratic cost of sorting is not the primary bottleneck.

7 Derivation of New Pareto Solutions from Existing Ones

Here we describe the code used to find new Pareto-optimal solutions by drawing lines between existing ones and interpolating solutions along those lines. This is much more time efficient than running the algorithm again, allowing the creation of new efficient solutions on par with the ones the algorithm develops, without having to wait for the full runtime of the algorithm.

7.1 Motivation

A full NSGA-III run is expensive, and the resulting Pareto front, typically around 30 non-dominated solutions in four-dimensional objective space, is sparse: there are large gaps between neighboring solutions that the algorithm never sampled or did not have time to visit. The idea behind the interpolation step is that two nearby Pareto-optimal layouts are likely to be "similar" in chromosome space as well as objective space, so a layout built from a blend of their two chromosomes may itself be a valid, non-dominated solution that fills one of those gaps. If so, we get a denser Pareto front made of solutions just as effective as the ones they are based on, without paying the computing cost of another full optimization run.

7.2 Ordering the Front

Interpolating "between neighbors" requires deciding which solutions are neighbors. We order the front by a nearest-neighbor traversal in normalized objective space: we start from one solution, then we repeatedly move to the closest unvisited solution, creating a path that goes through this Pareto front. Normalizing each objective to $[0, 1]$ before measuring distance prevents any single objective whose raw values are large from dominating the ordering.

7.3 Interpolation and Evaluation

For each consecutive pair of chromosomes $\mathbf{x}_a$ and $\mathbf{x}_b$ along the path, we generate several candidates by linear interpolation, rounding back to integers because the encoding is integer-valued:

$$\mathbf{x}_{\text{interp}} = \text{round}\big((1-t)\,\mathbf{x}_a + t\,\mathbf{x}_b\big), \qquad t = \tfrac{j}{n+1},\ j = 1, \ldots, n, \tag{11}$$

where n is the number of interpolation points requested per segment. Each interpolated chromosome is then decoded and evaluated just as it is in the main optimization, including the collision constraint, and any infeasible candidate is discarded.

7.4 Non-Dominance Filtering

A feasible candidate is kept if it is not dominated by any solution in the original front. The dominance check rejects a candidate with objective vector $\mathbf{f}$ if there exists an original solution $\mathbf{f}'$ that is no worse in every objective and strictly better in at least one. The candidates that pass this filter are new non-dominated solutions that the algorithm did not find on its own, and they are added to our Pareto front.

```python
def is_non_dominated(f_new, F_front):
    for f_existing in F_front:
        if np.all(f_existing <= f_new) and np.any(f_existing < f_new):
            return False
    return True
```

7.5 Results of Interpolation

In a run starting from 30 non-dominated solutions, interpolation discovered 37 additional non-dominated solutions, more than doubling the size of the front. Because each interpolated candidate requires only a single decode and evaluation, rather than a full run, this step is roughly two to three orders of magnitude cheaper than running the full algorithm, and it is about 600× cheaper than our baseline runs on a 20×20 grid with 500 generations and a population size of 120. This makes interpolation an effective way to add to the front after running the algorithm, and it becomes more effective the more generations the algorithm is run for.

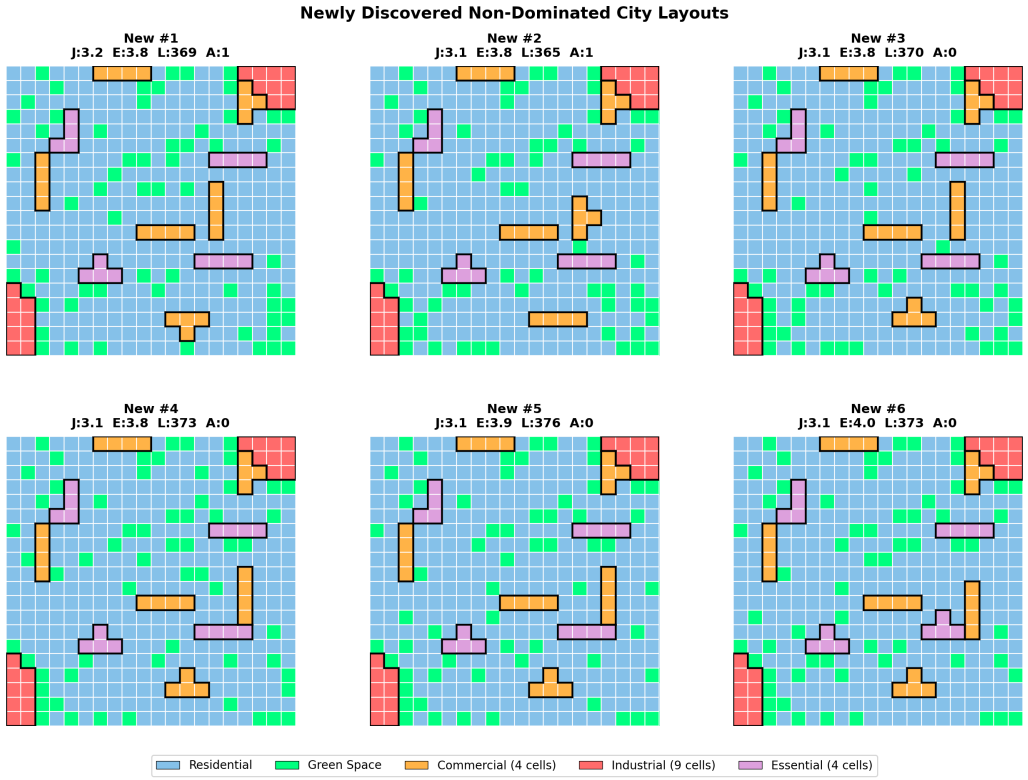


Figure 7: New non-dominated layouts discovered by interpolation. These solutions were never produced by the optimizer directly; each was decoded from a blend of two neighboring Pareto chromosomes.

Acknowledgment of AI Use

Anthropic's Claude [1] was used in the following capacities. The code was developed iteratively through a mixture of human and AI contribution: beyond the initial fraction-based test case, Claude assisted in the creation and/or debugging of the following code: the main NSGA-III implementation, the cluster-growth encoding, and the custom integer operators. Claude assisted in the creation of the interactive GUI described

in Section 4.5 and contributed to the Pareto front interpolation algorithm described in Section 7. Claude also assisted in the generation of the following figures: the taxicab distance calculation example (Figure 1), the NSGA-III flowchart (Figure 2), and the cluster-growth encoding diagram (Figure 3).

References

[1] Anthropic, "Claude," large language model, 2026. [Online]: `https://www.anthropic.com/claude`

[2] T. Sicuaio, P. Zhao, P. Pilesjö, A. Shindyapin, and A. Mansourian, "Sustainable and resilient land use planning: A multi-objective optimization approach," *ISPRS International Journal of Geo-Information*, vol. 13, no. 3, 2024.

[3] J. H. Holland, "Genetic algorithms," *Scientific American*, vol. 267, no. 1, pp. 66–73, 1992.

[4] C. A. Coello Coello, G. B. Lamont, and D. A. Van Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd ed. New York: Springer, 2007.

[5] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

[6] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[7] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," *IEEE Access*, 2020.

[8] I. Das and J. E. Dennis, "Normal-boundary intersection: A new method for generating the Pareto surface in nonlinear multicriteria optimization problems," *SIAM Journal on Optimization*, vol. 8, no. 3, 1998.

[9] E. F. Krause, *Taxicab Geometry: An Adventure in Non-Euclidean Geometry*. New York: Dover Publications, 1986.